POINT OF VIEW WHITE PAPER

The Agent Harness

Why the Orchestration Layer — Not the Model — Determines Agent Success

AUTHOR
Shyam Sridhar

DATE
March 2026

CLASSIFICATION
Public

LICENSE
MIT

DRAFT

Executive Summary

The AI industry is in the grip of a **model attribution error.** When an autonomous agent fails — hallucinating, looping, taking destructive action, or losing track of its task — the instinct is to blame the model. *The model needs to be smarter. The model needs more context. The model needs better alignment.*

But across research benchmarks, open-source agent projects, and the lived experience of engineering teams building agentic systems, a different pattern emerges: **the agent harness — the orchestration layer that wraps the model — is the primary determinant of whether an agent is reliable, safe, and useful.**

Two tools can use the same model and produce dramatically different outcomes because their harnesses differ in how they assemble context, validate tool calls, control loops, and enforce policy. The model is the reasoning engine. The harness is the operating system and control plane that makes the engine useful, safe, and repeatable.

This paper argues that engineering teams should shift investment from model selection to harness design. It defines the agent harness, decomposes its **five architectural pillars,** catalogs the most common failure modes, and provides architectural patterns drawn from Anthropic's research, Berkeley's work on compound AI systems, and open-source agent implementations.

SECTION 01

The Invisible Layer

A language model can produce text. It cannot, by itself, read files, run tests, open pull requests, apply policy checks, or decide when to stop working. The harness is what turns model output into actions with controlled side effects.

Yet in most conversations about AI agents, the harness is invisible. Discussion centers on model benchmarks, parameter counts, context window sizes, and training data. The orchestration substrate — how context is assembled, how tools are executed, how loops are controlled, how policies are enforced — is treated as implementation detail.

> *This is like evaluating a car engine without considering the transmission, brakes, steering, and chassis. The engine matters, but the vehicle's behavior depends on the entire system.*

The invisibility of the harness has a concrete cost. Engineering teams spend weeks evaluating models, running evals, and fine-tuning prompts when their actual reliability problems are:

- Unclear stopping rules that cause infinite loops
- Weak tool validation that allows malformed calls
- Poor context budgeting that fills the window with irrelevant data
- Missing feedback loops that prevent self-correction

DRAFT

These are harness design problems, not model capability problems.

DRAFT

SECTION 02

Defining the Agent Harness

An agent harness is the runtime that constructs context, executes tool calls, enforces guardrails, and decides when each loop iteration should continue or stop.

It is not:

- A chat UI wrapper
- A prompt template library
- A framework's decorator syntax
- An API client for a language model

It is:

The execution substrate that determines whether an agent can operate safely and reliably over many steps. The distinction is architectural, not cosmetic.

ARCHITECTURE — THE HARNESS IN CONTEXT

SECTION 03

The Five Pillars of a Production Harness

A production-grade agent harness stands on five pillars. Remove any one and the system collapses — not eventually, but within the first multi-step task. Each pillar enforces a specific architectural guarantee, has a measurable implementation contract, and produces a known failure signature when absent.

DRAFT

PILLAR 01

Context Assembly

Guarantee: *Every model invocation receives a token payload that maximizes comprehension per token spent — system prompt, harness instructions, project context, tool definitions, and dynamic state — structured so the model knows what exists, what it can do, and what the user expects.*

This is not "write a system prompt." This is an allocation problem. A 200K-token context window is a fixed resource budget. Every byte of irrelevant context displaces relevant context.

Layer	Content	Injection Point
Static foundation	Role identity, safety rules, output format constraints	System prompt, set once
Project context	Code conventions, architecture doc, dependency map	Loaded from AGENTS.md at session start
Dynamic state	Current file tree, git status, in-progress task state	Injected per turn by the harness
Tool definitions	Name, description, parameter schema, usage examples	Registered at harness initialization
User message	The actual task or follow-up	Appended last (queries at bottom improve quality up to 30%)

FAILURE SIGNATURE

Context starvation. The model doesn't know what resources exist, what tools are available, or what vocabulary the user expects. The model isn't confused — it's uninformed.

DRAFT

PILLAR 02

Tool Integrity (The Agent-Computer Interface)

Guarantee: *Every tool call is schema-validated before execution, runs in a controlled environment, returns output the model can reason about, and produces actionable error messages that enable self-correction.*

> *"Anthropic spent more time optimizing tools than the overall prompt."*
>
> — Anthropic SWE-bench Team

Format	Model Cost	When to Use
Full file content	High token count, unambiguous	File reads under 500 lines
Diffs	Error-prone for model tracking	Avoid as model output; acceptable as harness display
Structured JSON	Escaping burns tokens	API responses, structured data
Plain text summary	Low token count, high comprehension	Verbose tool results

FAILURE SIGNATURE

Tool misuse. The model calls tools with wrong parameter types, nonsensical argument combinations, or in illogical sequences. Root cause: ambiguous tool definitions, not model capability.

DRAFT

PILLAR 03

Loop Discipline

Guarantee: *The agent's execution cycle uses explicit signals — not heuristics — to determine when to continue, retry, or halt.*

THE TOOLRESULT CONTRACT

success	shouldContinue	Meaning
true	true	Tool worked. Keep pursuing the goal.
true	false	Task complete. Stop the loop.
false	true	Tool failed. Try another approach.
false	false	Unrecoverable. Halt immediately.

Stopping Condition	Trigger	Harness Action
Explicit completion	Model calls complete_task	Stop loop, return summary
Iteration ceiling	Turn count exceeds maxIterations	Stop loop, return partial results
Token budget	Context utilization exceeds 80%	Trigger compaction or spawn sub-agent
Safety halt	Policy violation detected	Stop immediately, log, alert
Idle detection	N turns with no tool calls	Prompt model: "Are you blocked?"

FAILURE SIGNATURE

The infinite loop. The agent runs for hundreds of turns because nobody told it how to stop. Both infinite loops and premature termination are harness bugs.

DRAFT

PILLAR 04

Policy Enforcement

Guarantee: *Every tool call passes through a permission gate before execution — classified as allow, deny, or confirm — so that no side effect occurs without matching the harness's policy matrix.*

THE PERMISSION MATRIX

Action Category	Policy	Examples
Read-only	ALLOW	read_file, list_dir, git status
Scoped write	ALLOW	write to output/, working branch
Modifying source	CONFIRM	edit src/*, modify package.json
Destructive	CONFIRM	git push --force, rm -rf, DROP
External-facing	CONFIRM	Post to Slack, deploy to prod
Credential access	DENY	Read .env, access secrets mgr

FAILURE SIGNATURE

Destructive actions. The agent deletes files, force-pushes to main, or posts to production channels. The instinct is to blame judgement. The actual cause: nobody built the gate.

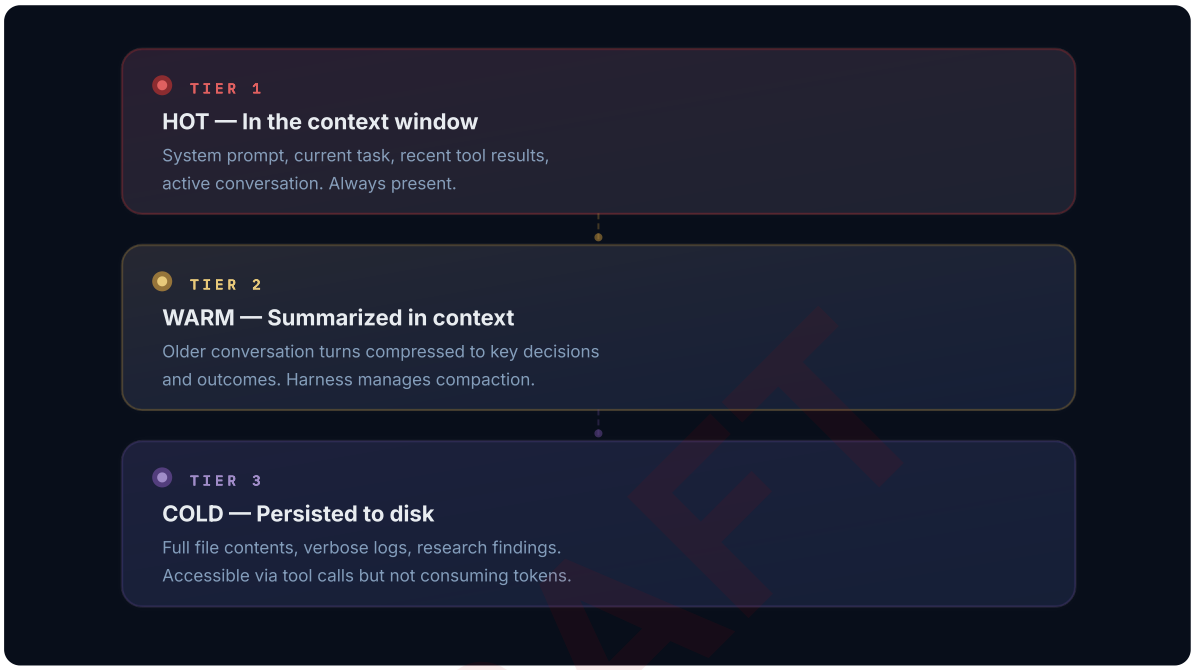
DRAFT

PILLAR 05

Context Lifecycle

Guarantee: *The context window is actively managed as a finite, depletable resource — with explicit rules for what stays hot, what gets compacted, what gets persisted to disk, and when work is offloaded to sub-agents.*

THREE-TIER MEMORY ARCHITECTURE



Mechanism	What It Does	When to Trigger
Compaction	Summarize older messages, drop verbose results	Context utilization exceeds 80%
Progress file	Agent writes progress.md with current state	Every 5 turns, before compaction
Sub-agent delegation	Spawn sub-agent with own context window	Subtask would consume >20% of remaining context
context.md pattern	Persistent file read at session start	Multi-session tasks

FAILURE SIGNATURE

DRAFT

Context exhaustion. The agent starts producing incoherent responses or losing track of its objective around turn 15. The diagnosis is “the model forgot” — the actual cause is that the harness let critical information get displaced by noise.

DRAFT

SECTION 04

The Attribution Error

The most consequential insight from studying agent failures is that **most reliability problems blamed on "the model" are harness design problems.**

Symptom	Misattribution	Actual Cause
Infinite loops	"Model can't figure out when to stop"	No explicit completion mechanism in the harness
Context exhaustion	"Context window isn't big enough"	Harness dumps full outputs without summarization
Destructive actions	"Model doesn't understand consequences"	No policy layer — no actions require confirmation
Tool misuse	"Model doesn't understand the tools"	Vague tool descriptions, ambiguous parameters
No self-correction	"Model can't recognize mistakes"	Harness doesn't feed actionable errors back into the loop

KEY INSIGHT

Each failure mode above shares a common structure: a model behavior that *looks like* a capability gap but is actually caused by missing harness infrastructure. Fixing the harness — not switching models — resolves the problem.

DRAFT

SECTION 05

Harness Patterns for Production

The Augmented LLM as Foundation

Every agentic system starts with an LLM enhanced with three capabilities: retrieval, tools, and memory. These capabilities are harness-level additions. The model itself doesn't "have" tools — the harness gives it tools, validates their use, and executes their results.

Workflow Patterns (Deterministic Control Flow)

Pattern	Harness Responsibility	Best For
Prompt Chaining	Manages sequential model calls, validates intermediate results	Tasks with known subtasks
Routing	Classifies inputs, directs to specialized handlers	Distinct input categories
Parallelization	Runs concurrent model calls, aggregates results	Independent subtasks

Agent Patterns (Model-Driven Control Flow)

Pattern	Harness Responsibility	Best For
Orchestrator-Workers	Central model decomposes task; harness spawns workers	Complex tasks with unpredictable subtasks
Evaluator-Optimizer	One model generates; another evaluates; harness loops	Tasks with clear evaluation criteria
Autonomous Loop	Harness runs model in a loop with full tool access	Open-ended problems

> *"Despite handling sophisticated tasks, agents are typically just LLMs using tools based on environmental feedback in a loop."*
>
> — Anthropic, "Building Effective Agents"

DRAFT

The Unified Orchestrator Pattern

One execution engine, many agent types. All agents use the same harness with different configurations: different tools, different system prompts, different model tiers, different iteration limits. This separates the pillars cleanly: the harness handles execution mechanics; the configuration handles agent identity.

DRAFT

SECTION 06

Compound Harnesses: Multi-Agent Orchestration

When a system goes from one agent to many, the importance of harness design increases non-linearly. Each sub-agent needs its own context assembly, loop discipline, and policy boundary.

Single-Agent Pillar	Compound Equivalent
Context assembly	Per-agent context scoping + shared workspace
Tool integrity	Cross-agent tool access controls
Loop discipline	Dependency-aware scheduling across agents
Policy enforcement	Hierarchical permission boundaries
Context lifecycle	Shared state management across instances

Swarm Orchestration Primitives

- **Teams:** Named groups of agents with a leader and teammates. The harness manages team membership, message routing, and shutdown coordination.
- **Inboxes:** Asynchronous message channels between agents. The harness serializes and deserializes messages, handles delivery, and prevents deadlocks.
- **Task Dependencies:** Some agent tasks depend on outputs from others. The harness resolves dependencies, schedules execution order, and propagates failures.
- **Spawn Backends:** Sub-agents can run in-process, in separate terminal panes, or on remote machines. The harness abstracts the spawn mechanism.

DRAFT

Resource Allocation

A non-obvious harness responsibility: how to distribute budget (latency, cost, compute) across components. If the total budget is 100ms, should the retriever get 20ms and the model 80ms, or the reverse? Should the orchestrator use the powerful model tier while workers use the fast tier? These are harness-level decisions with first-order impact on system behavior.

SECTION 07

Design Principles

1. **Simplicity Over Sophistication.** Start with the simplest harness that enforces the five pillars. Add complexity only when measurement proves it improves outcomes. A harness that clearly implements the five pillars in readable, debuggable code beats an abstraction-heavy framework every time.

2. **Transparency Over Opacity.** Surface the agent's state at every point in its loop: which tools are being called, what context is being assembled, what policies are being applied, where in the loop the agent currently is.

3. **ACI-Native Tool Design.** Treat tool design with the same rigor as UX design. Clear descriptions. Obvious parameter names. Documented edge cases. Error messages that enable self-correction.

4. **Explicit Over Heuristic.** Prefer explicit signals over heuristic detection at every harness decision point. Explicit completion tools over inferred stopping. Explicit permission checks over assumed safety.

5. **Feedback Loops Over One-Shot Execution.** The value of an agent is not one perfect model response. It is a controlled loop that can self-correct with runtime feedback.

6. **Atomic Tools, Compound Behavior.** Tools should be primitive capabilities: read a file, write a file, run a command. Features are outcomes the agent achieves by composing primitives in a loop.

DRAFT

SECTION 08

From Theory to Implementation

This paper did not start as theory. It grew out of building agentic systems — knowledge graph pipelines that hallucinated temporal relationships, LLM defense systems where prompt injection revealed how little the harness was doing to protect the model, and the iterative process of designing orchestration layers that actually work.

Sharkbait — an open-source CLI coding assistant built on Bun and TypeScript — became the project where these lessons converged into a deliberate harness architecture. It is not presented here as an exemplary system but as an honest implementation of the five pillars.

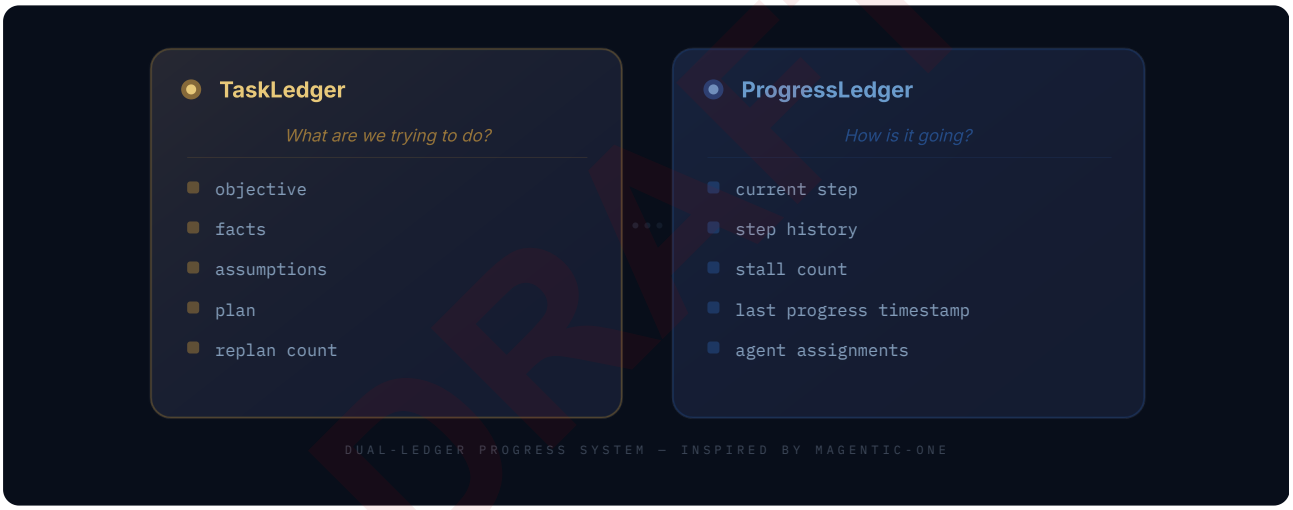
Pillar Mapping: How the Five Guarantees Manifest in Code

Context Assembly. Sharkbait's ContextManager separates context into two categories: PreservedContext (system prompt, task ledger, recent messages, active files, error context — never compacted) and CompactableContext (older messages, tool results — eligible for summarization).

TYPESCRIPT — PRESERVEDCONTEXT INTERFACE

```
interface PreservedContext {
  systemPrompt: string;
  taskLedger: TaskLedger;
  recentMessages: Message[];
  activeFiles: FileContext[];
  errorContext: ErrorContext[];
}
```

Loop Discipline. The AgentLoop implements a dual-ledger progress system inspired by Microsoft Research's Magentic-One. Rather than a single loop counter, it separates **intent** from **execution**:



This separation enables a three-state progression: **continue** → **replan** → **escalate** — extending the whitepaper's two-state model.

TYPESCRIPT — LIFECYCLE HOOKS

DRAFT

```
type HookType =
  | "PreToolUse" | "PostToolUse"
  | "Stop" | "OnError"
  | "PreAgentSwitch" | "PostAgentSwitch"
  | "OnTokenLimit" | "OnUserInput"
  | "SessionStart" | "SessionEnd";
```

Where the Harness Remains Incomplete

- **Parity discipline.** The action parity audit has not been formalized for CLI workflows.
- **Dynamic context injection** of project-specific vocabulary definitions.
- **Self-modification.** Agents cannot yet modify their own prompts based on feedback — a capability that would close the improvement-over-time loop.

These gaps are the next engineering frontier, not oversights.

SECTION 09

Calling the Question

The model will improve. Next year's model will be meaningfully better than this year's. But the harness is where engineering teams can create durable competitive advantage *right now.*

DRAFT

The Investment Case

Investment Area	Payoff Timeline	Durability
Model selection	Immediate but transient	Low — next release resets the advantage
Prompt engineering	Near-term	Medium — prompts need updating as models change
Harness architecture	**Near-term to long-term**	**High — the five pillars persist regardless of model**
Tool design (ACI)	Near-term	**High — good tools work with any model**
Policy framework	Ongoing	**High — safety requirements only increase**
Context lifecycle strategy	Near-term	**High — context management remains critical**

Recommended Actions

1. **Name the harness.** If your team builds agents and doesn't have an explicit concept of "the harness" — with owners, design documents, and review processes — you're operating on implicit assumptions. Make them explicit.
2. **Audit the five pillars.** For every agent in production, ask: How is context assembled? How are tools validated? How does the loop stop? What policies gate destructive actions? How is the context window managed?
3. **Invest in tool design.** Anthropic spent more time optimizing tools than prompts for their benchmark agents. Most teams do the opposite. Reverse the ratio.
4. **Separate harness from configuration.** Use the unified orchestrator pattern: one execution engine, many agent types.
5. **Build feedback loops, not guardrails.** The goal is a loop where mistakes are detected, surfaced with enough information for self-correction, and retried — within well-defined policy boundaries.

DRAFT

6 **Treat harness design as a first-class engineering discipline.** It is not plumbing. It is the system that determines whether your $20/hour reasoning engine produces $200/hour outcomes or $2/hour confusion.

DRAFT

CONCLUSION

The Most Important, Most Neglected Component

The agent harness is the most important and most neglected component of agentic AI systems. While the industry focuses on model capability — larger context windows, better benchmarks, more training data — the actual determinant of agent reliability in production is the orchestration substrate: how context is assembled, how tool integrity is maintained, how loop discipline is enforced, how policies gate side effects, and how the context lifecycle is managed across turns.

Two systems with the same model produce dramatically different outcomes when their harnesses differ. The model is necessary but not sufficient. The harness is where engineering judgment, system design, and operational rigor create durable, compounding advantage.

> *The question for engineering leaders is not "which model should we use?"*
> *It is: **"how good is our harness?"***

REFERENCES

Sources and Further Reading

1. **Anthropic.** "Building Effective Agents." Anthropic Research, 2024. — Agent architecture patterns, ACI design, tool optimization, deployment guidance.
2. **Zaharia, M. et al.** "The Shift from Models to Compound AI Systems." Berkeley Artificial Intelligence Research (BAIR), February 2024. — Compound system architectures, optimization challenges, production considerations.
3. **Anthropic.** "Prompting Best Practices for Claude 4.6." Anthropic Platform Documentation, 2026. — System prompt design, long-horizon reasoning, context management.
4. **Anthropic.** "Agent-Native Architecture." Compound Engineering Plugin, 2025–2026. — Parity, granularity, composability, emergent capability, improvement-over-time principles.
5. **Anthropic.** "Agent Execution Patterns." Compound Engineering Plugin, 2025–2026. — Completion signals, partial completion, context limits, unified orchestrator pattern.
6. **Anthropic.** "Claude Code Swarm Orchestration." Compound Engineering Plugin, 2025–2026. — Multi-agent primitives, team coordination, spawn backends, messaging protocols.
7. **Microsoft Research.** "Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks." Microsoft Research, 2024. — Dual-ledger progress tracking, orchestrator-worker architecture.
8. **Sridhar, S.** "Sharkbait: CLI Coding Assistant." GitHub, 2026. — Open-source implementation of the five pillars, dual-ledger loop discipline, Beads git-native persistence.

DRAFT

This white paper was co-authored using Claude's compound engineering plugin for research synthesis and the brainstorming skill for argument development. The agent harness definition and pillar framework are adapted from an original concept document by the author. The case study in Section 8 draws from the author's open-source work on Sharkbait.

DRAFT

The question for engineering leaders is not "which model should we use?"

It is: "how good is our harness?"

DRAFT

MIT LICENSE · MARCH 2026 · GITHUB.COM/SHYAMSRIDHAR123/HARNESS

DRAFT